Light Class activity: FAST API

YKumar, 2024-6-20

Based on https://www.youtube.com/watch?v=3vfum74ggHE

Contents

1. Meet the FAST API

Create a folder fastapi, open it in VSC and open a new terminal

Observe: https://fastapi.tiangolo.com/ **page**

2: Always be coding with ChatGPT

Talke screenshots from the video FAST API part (5-6 min)

Yes, I can see the images. They provide instructions for setting up a virtual environment, activating it, installing FastAPI, and installing additional packages. Here's the text extracted from the images to ensure clarity:

Yes, I can see the images. They provide instructions for setting up a virtual environment, activating it, installing FastAPI, and installing additional packages. Here's the text extracted from the images to ensure clarity:

2.1. Create a new virtual environment for this project

python3 -m venv venv

2.2. Activate it

On Windows:

venv\Scripts\activate

On Unix or MacOS:

source venv/bin/activate

```
PS C:\Users\galae\Desktop\00472-337513824 GS
res\3320\fastapi> python3 -m venv venv
PS C:\Users\galae\Desktop\00472-337513824 GS
res\3320\fastapi> .\venv\Scripts\activate
(venv) PS C:\Users\galae\Desktop\00472-33751
```

2.3. Install FastAPI

pip install fastapi

2.4. Install additional packages

Install server to run it on:

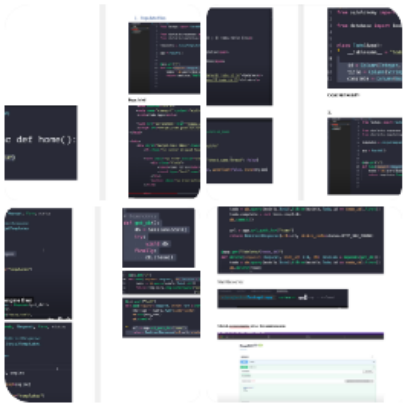
pip install "uvicorn[standard]"

For database support, install:

pip install python-multipart sqlalchemy jinja2

2.5. Create Files

Would you like to proceed with any additional instructions or modifications?



yes I need all this code too with comments , make sure nothing is missing, fill out the gaps

Here is the complete FastAPI to-do app with comments to fill in the gaps from the images you provided:

Create the necessary files and directory structure

4.1. Create the following files and directories:

app.py
database.py
models.py
templates directory

5. Populate the files

app.py

from fastapi import FastAPI, Depends, Request, Form, status

```
from starlette.responses import RedirectResponse
from starlette.templating import Jinja2Templates
from sqlalchemy.orm import Session
import models
from database import SessionLocal, engine

# Initialize the app
app = FastAPI()

# Set up Jinja2 templates
templates = Jinja2Templates(directory="templates")

# Create the database tables
models.Base.metadata.create_all(bind=engine)

# Dependency to get the database session
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/")
async def home(request: Request, db: Session = Depends(get_db)):
    todos = db.query(models.Todo).all()
    return templates.TemplateResponse("base.html", {"request": request, "todo_list":
todos})

@app.post("/add")
async def add(request: Request, title: str = Form(...), db: Session =
Depends(get_db)):
    new_todo = models.Todo(title=title)
    db.add(new_todo)
    db.commit()
    url = app.url_path_for("home")
    return RedirectResponse(url=url, status_code=status.HTTP_303_SEE_OTHER)

@app.get("/update/{todo_id}")
async def update(request: Request, todo_id: int, db: Session = Depends(get_db)):
    todo = db.query(models.Todo).filter(models.Todo.id == todo_id).first()
    todo.complete = not todo.complete
```

```
    db.commit()
    url = app.url_path_for("home")
    return RedirectResponse(url=url, status_code=status.HTTP_302_FOUND)

@app.get("/delete/{todo_id}")
async def delete(request: Request, todo_id: int, db: Session = Depends(get_db)):
    todo = db.query(models.Todo).filter(models.Todo.id == todo_id).first()
    db.delete(todo)
    db.commit()
    url = app.url_path_for("home")
    return RedirectResponse(url=url, status_code=status.HTTP_302_FOUND)
```

database.py

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

SQLALCHEMY_DATABASE_URL = "sqlite:///./db.sqlite"

engine = create_engine(SQLALCHEMY_DATABASE_URL,
connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

models.py

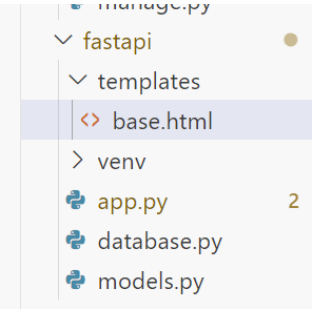
```
from sqlalchemy import Boolean, Column, Integer, String
from database import Base

class Todo(Base):
    __tablename__ = "todos"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    complete = Column(Boolean, default=False)
```

Create the HTML templates

templates/base.html




```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gryffindor To-Do App</title>
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/semantic-
ui@2.4.2/dist/semantic.min.css">
</head>
<body>
    <div class="ui menu">
        <div class="header item">Gryffindor To-Do App</div>
        <a class="item" href="/">Home</a>
    </div>
    <div class="ui container">
        {% block content %}{% endblock %}
    </div>
</body>
</html>
```

templates/index.html

```
{% extends 'base.html' %}

{% block content %}
<h1 class="ui header">To-Do List</h1>
<form class="ui form" action="{{ url_for('add') }}" method="post">
    <div class="field">
        <label>Todo Title</label>
        <input type="text" name="title" placeholder="Enter Todo...">
    </div>
    <button class="ui button" type="submit">Add</button>
```

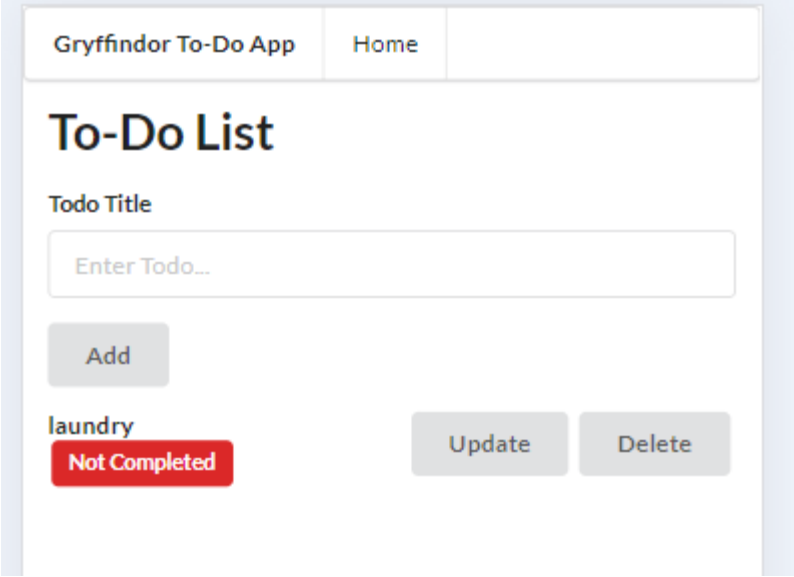
```
</form>
<div class="ui divided list">
    {% for todo in todo_list %}
    <div class="item">
        <div class="right floated content">
            <a class="ui button" href="{{ url_for('update', todo_id=todo.id)
}}">Update</a>
            <a class="ui button" href="{{ url_for('delete', todo_id=todo.id)
}}">Delete</a>
        </div>
        <div class="content">
            <div class="header">{{ todo.title }}</div>
            {% if todo.complete %}
            <span class="ui green label">Completed</span>
            {% else %}
            <span class="ui red label">Not Completed</span>
            {% endif %}
        </div>
    </div>
    {% endfor %}
</div>
{% endblock %}
```

7. Start the server

uvicorn app:app –reload

```
N_Lectures\3320\fastapi> uvicorn app:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\galae\
\Desktop\\00472-337513824 GS Backup 12.20.22 JC\\KEAN_Lectures\\3320\\fast
api']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [17588] using WatchFiles
```

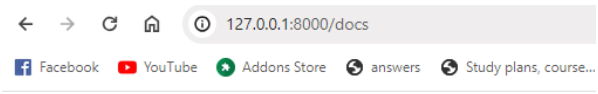
Follow the link

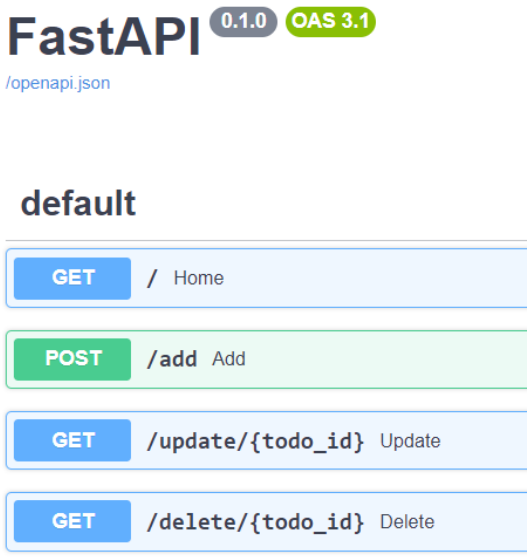


Verify you populated all files, debug as needed

8. Check automatic documentation

Navigate to `http://127.0.0.1:8000/docs` to view the interactive API documentation generated by FastAPI.





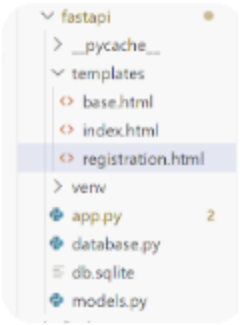
This completes the setup for a Gryffindor-themed to-do app using FastAPI. The provided code includes comments and ensures all required functionality is implemented and explained.

1. Improve with Gemini

https://gemini.google.com/

here are my models.py from sqlalchemy import Boolean, Column, Integer, String from database import Base class Todo(Base): __tablename__ = "todos" id = Column(Integer, primary_key=True, index=True) title = Column(String, index=True) complete = Column(Boolean, default=False) and GUI {% extends 'base.html' %} {% block content %} <h1 class="ui header">To-Do List</h1> <form class="ui form" action="{{ url_for('add') }}" method="post"> <label>Todo Title</label> <input type="text" name="title" placeholder="Enter Todo..."> <button class="ui button" type="submit">Add</button> </form> {% for todo in todo_list %}   Update Delete   {{ todo.title }} {% if todo.complete %} Completed {% else %} Not Completed {% endif %}   {% endfor %} {% endblock %} can you change this to do to student course registration?

any other pages should be changed too?

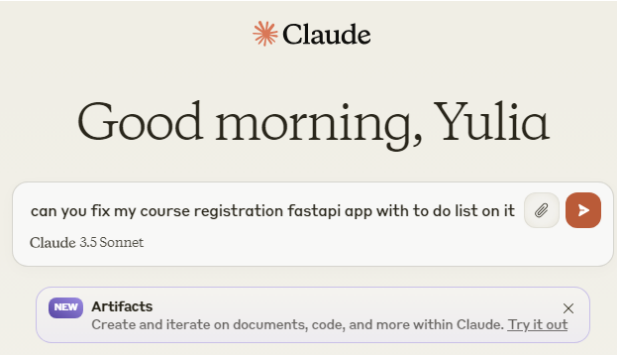


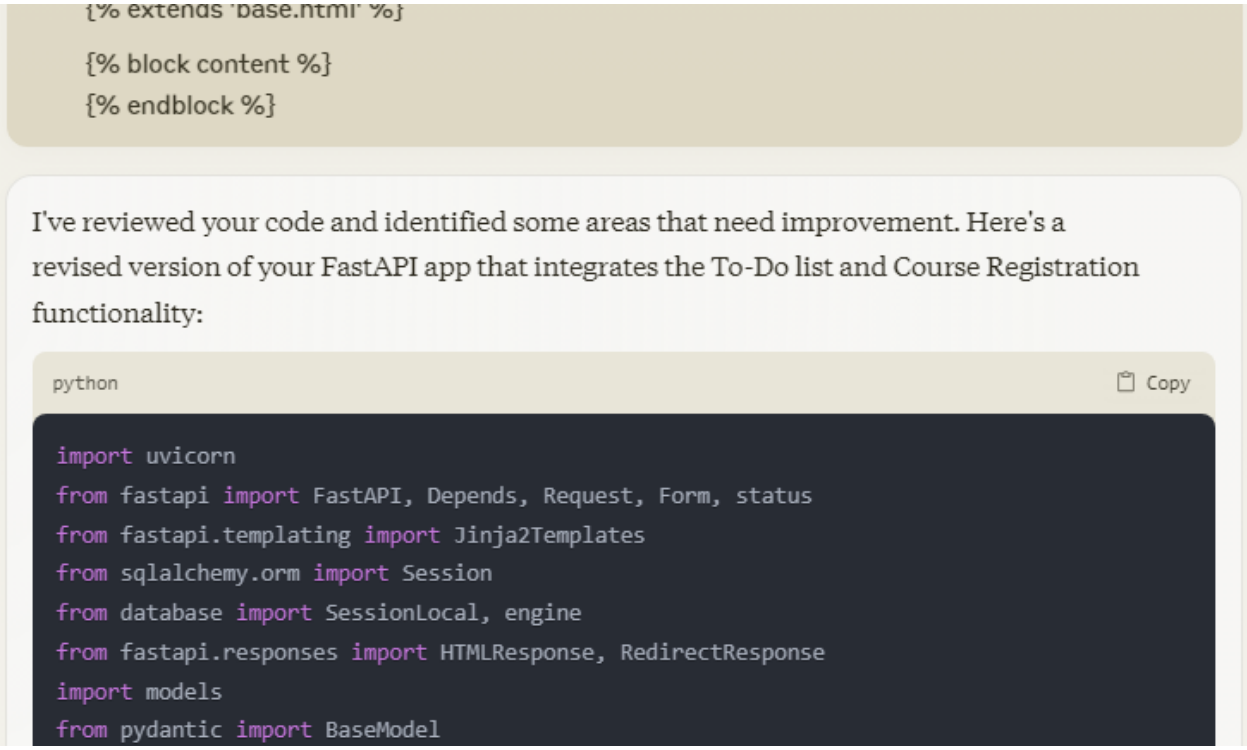
ok provide code for all files used

Try on your own

2. Add some Claude

https://www.anthropic.com/claude





3. ChatGPT to the rescue